[code_analysis] Semantic check - Name Resolution

Created By	Hyung-Gyu Ryoo
Email	hgryoo@cubrid.com
Tags	코드분석
Team	개발2팀
Version	0.9

Name resolution Overview
SPEC 정보 생성
- **flat_entity_list**

SPEC 정보 연결 (name binding)
- 주요 동작 설명
- 간단한 예시
- Resolving *
- Derived table
- Subquery
- Join query
- oracle style outer join

Group by / Having Alias
Natural join resolution
FOR UPDATE 절 flag 설정

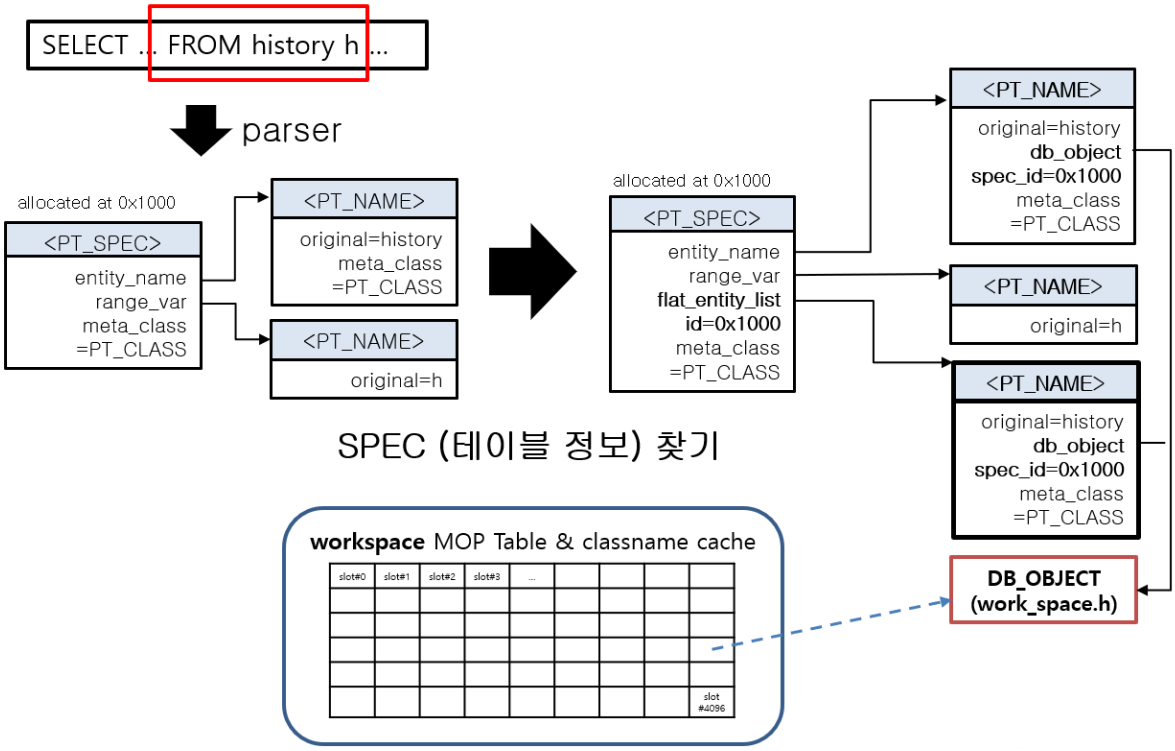
Name resolution Overview

Semantic check 에서 수행하는 단계 중 하나인 Name resolution 에서는 파싱트리가 포함하는 모든 PT_NAME에 대해 의미를 분석한다. PT_NAME에 대해 의미를 분석한다는 것은 이것이 테이블의 이름인지, 테이블의 별명인지, 컬럼의 이름인지 등의 것을 알아내고 그 의미 정보를 노드에 저장하는 것이다.

Name resolution는 다음의 순서로 수행한다.

1. **SPEC 정보 생성:** Spec 정보(PT_SPEC) 에 연결되어 있는 PT_NAME을 분석하고 이것으로부터 테이블 정보를 연결한다
2. **SPEC 정보 연결:** Spec 정보 이외의 모든 PT_NAME에 대해 적절한 table spec으로 연결한다
3. **Group by/Having Alias 연결:** Group by/Having 절의 alias를 연결한다
4. **Natural join resolution:** Natural join을 inner/outer join으로 변환한다. 이때 필요한 PT_NAME을 생성한다
5. **FOR UPDATE flag:** FOR UPDATE 구문에 해당하는 Spec 정보에 flag를 표시한다

SPEC 정보 생성



이 단계에서 전체 파싱 트리를 순회하며 PT_SPEC 노드와 이 노드를 포함한 상위 노드만을 확인하며 다음의 세 가지 정보를 생성한다.

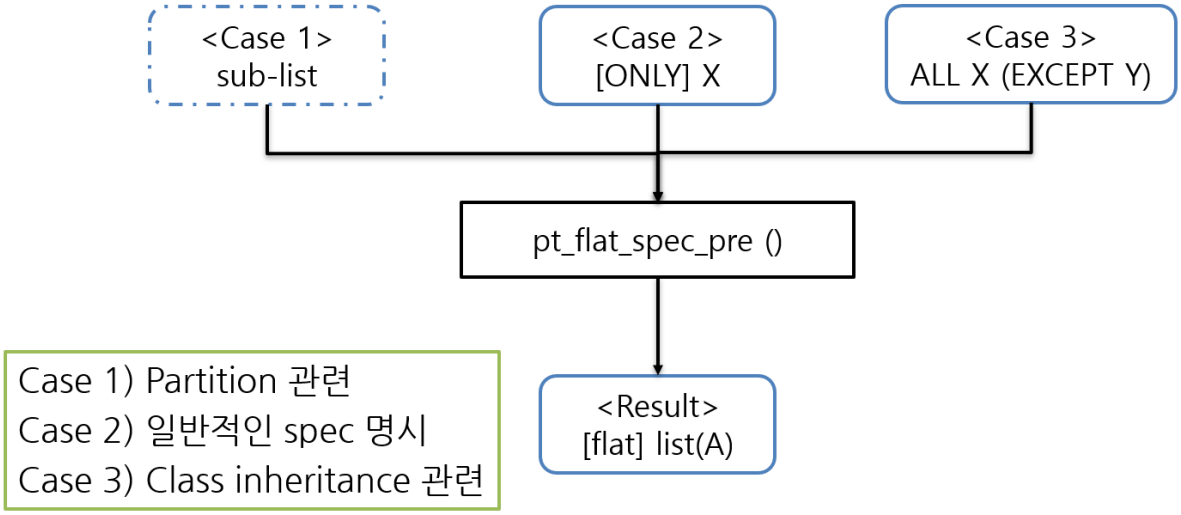
- **id, spec_id:** id 또는 spec_id는 PT_NAME이 어떤 PT_SPEC과 연결되는지를 저장하기 위해 사용하는 정보이다. PT_SPEC은 노드가 메모리에 할당된 주소를 id로 저장한다. 그리고 PT_NAME은 자신이 포함된 PT_SPEC 노드의 id를 spec_id로 저장한다.
- **db_object:** prefetch 단계에서 가져온 테이블 정보가 저장된 workspace로부터 DB_OBJECT 객체를 가져와 연결한다.
- **flat_entity_list:** PT_SPEC의 flat_entity_list에 PT_NAME 노드가 새로 생성된다. 이 노드는 entity_name에 연결된 노드와 동일한 값을 가질수도 있지만, 같은 객체는 아니다. flat_entity_list가 왜 필요한지 또 어떤 경우에 여러 PT_NAME 노드가 링크드 리스트로 연결되는지 다음에서 설명한다.

flat_entity_list

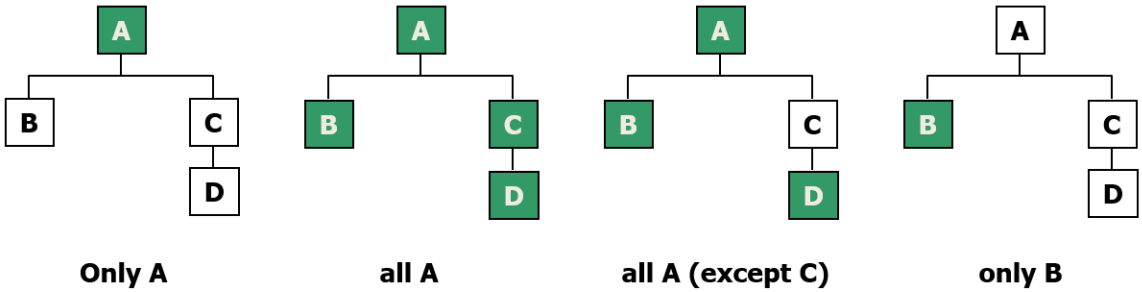
```
...
FROM
<table_specification>
...
```

```
<table_specification> ::= <single_table_spec> [<correlation>] | ...

<single_table_spec> ::= [ONLY] <table_name> |
                        ALL <table_name> [ EXCEPT <table_name> ]
```



flat_entity_list는 partition table이나 테이블의 상속을 처리하기 위해 생성한다. SQL 구문에서 명시한 spec의 이름은 entity_name에 연결되지만, partition table이나 상속 관계가 있는 테이블으로 ALL .. EXCEPT 구문을 사용하는 경우에는 entity_name이 가리키는 테이블과 연결된 다른 테이블의 정보도 함께 가져와야한다.



Queries)

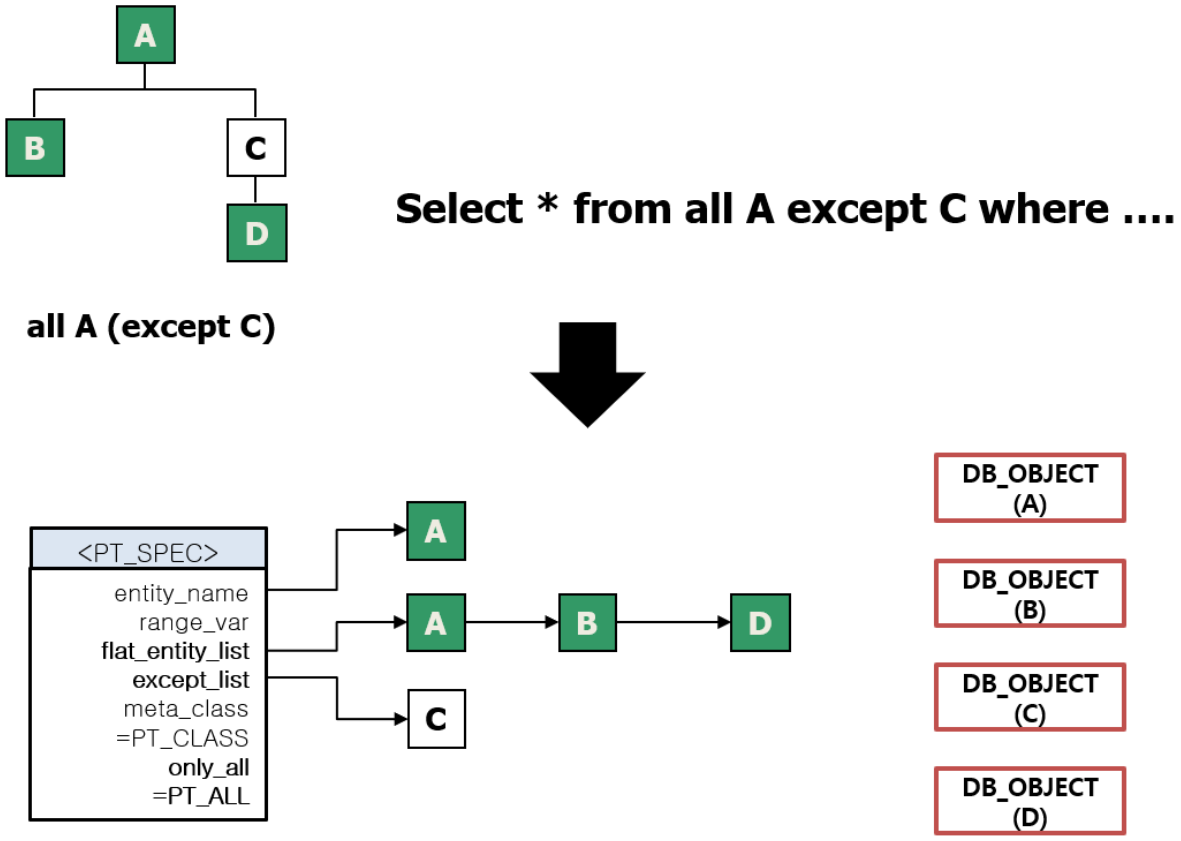
Select * from A where ....

Select * from all A where ....

Select * from all A except C where ....

Select * from only B where ...

위 그림은 ALL ... EXCEPT 구문의 사용 예시이다. 상속 관계가 있는 테이블로부터 원하는 테이블을 선택하여 질의할 수 있다.



위와 같은 예시에서 A, C는 질의에 명시되어있어 entity_name과 except_list에 연결되어 있지만 B, D 테이블에 대한 정보도 가져와 연결해야한다. 따라서 A, B, D에 대한 PT_NAME을 새로 생성하고 각각의 테이블 정보인 DB_OBJECT를 workspace로부터 가져와 연결한다.

소스 참조: pt_flat_spec_pre()

SPEC 정보 연결 (name binding)

주요 동작 설명

이 단계에서 전체 **파싱 트리에 대해 순회하면서 PT_NODE의 타입에 따라 조건에 맞는 동작을 수행**한다. 이 단계의 코드는 복잡하고 긴 로직으로 구현되어 있다. 그러나 실제로는 구문별로 고려하는 부분이 아니면 실행되지 않는다.

주로 수행하려는 동작은 다음의 두 가지를 목표로 한다.

(1) 파싱 트리에 존재하는 PT_NAME 노드에 대해서 PT_SPEC 노드의 정보와 연결(binding) 하는 것

(2) PT_NAME을 만들어내야 하는 구문의 경우 조건에 따라 파싱 트리를 수정하여 새로운 PT_NAME 노드를 만들거나 위치를 옮기는 것

SELECT **h.name**, **h.desc**, **h.year** FROM **history h** WHERE **h.year** = ?

해석해야 할 이름들

- history → 테이블 이름
- h → 테이블 별명
- h.name
- h.desc
- h.year

} 컬럼 이름

위 그림과 같이 구문에 등장하는 모든 이름(PT_NAME)에 대해서 어떤 의미를 가지는지해석하는 단계가 (1)이다.

(2)가 수행되는 상황은 구문별로 다양하다. 대부분의 복잡한 로직들은 구문별로 (2)를 수행하기 위한 목적으로 구현되어 있다. 다음은 새로운 PT_NAME 노드가 만들어지는 몇 가지 상황의 예시이다. 각 예시에 대해 자세히 설명하지는 않으니 참고한다.

- SELECT list에 * 또는 <table_name>.* 가 존재하는 경우 PT_NAME 노드의 리스트로 변환

- derived table

- CTE

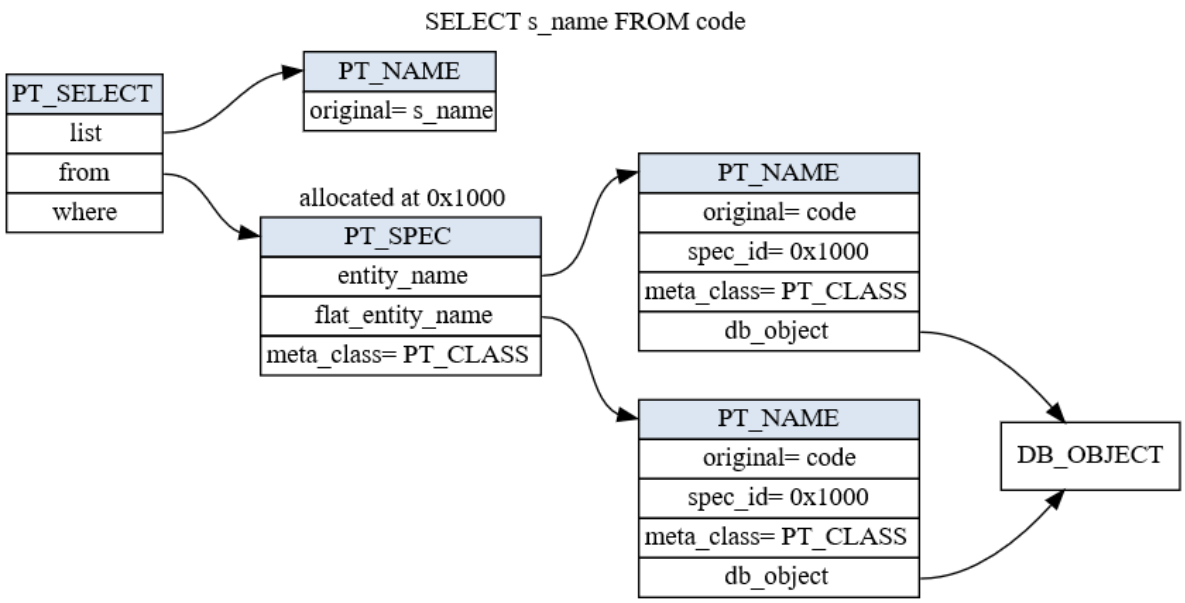
- Oracle style outer join

간단한 예시

이제 가장 단순한 예시부터 이 단계에서 수행하는 동작을 이해해보려고 한다.

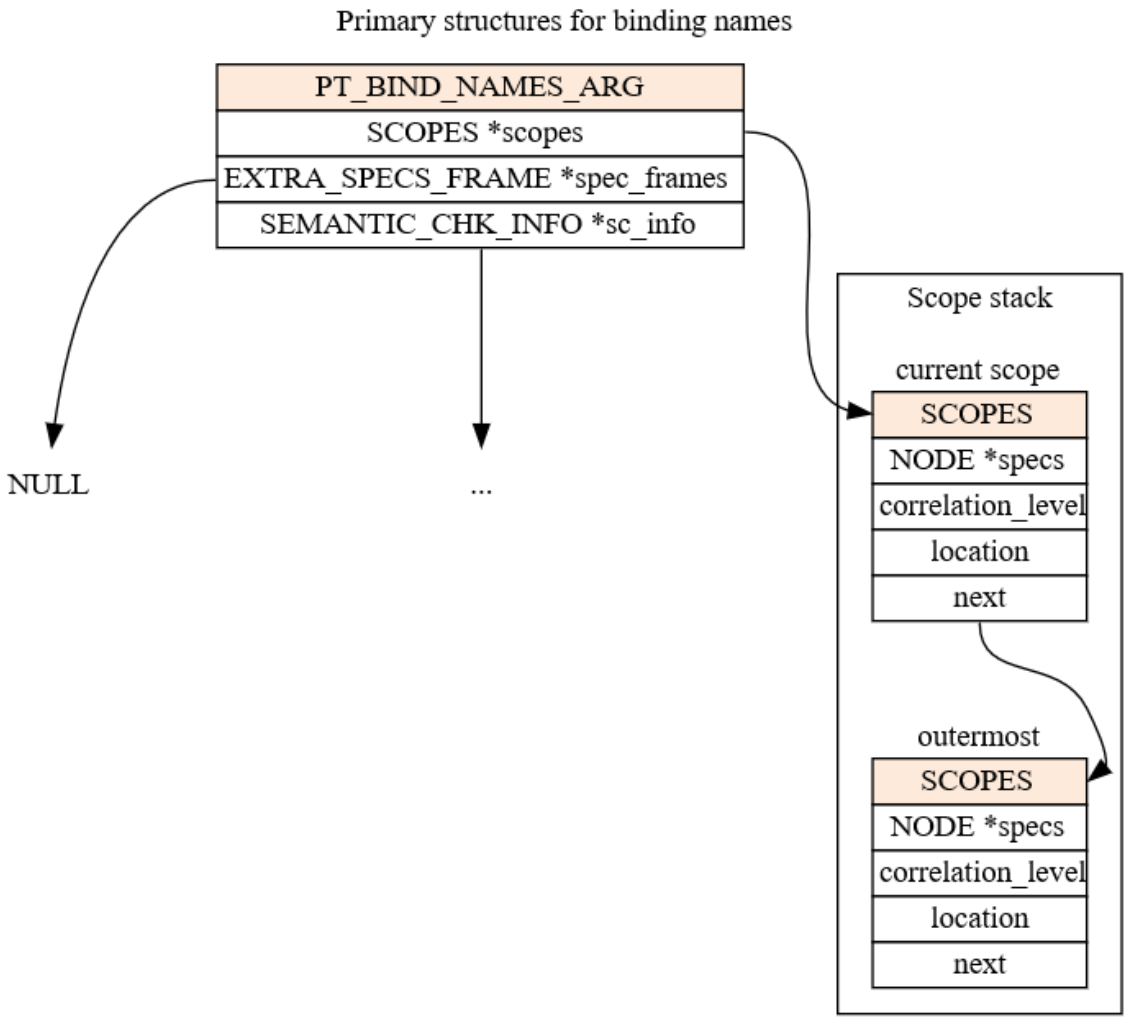
```
// example 1
SELECT s_name FROM code;
```

example 1에서 수행할 동작은 단 하나다. s_name를 code라는 이름에 연결하는 것이다. 만약 code 테이블에 s_name이라는 이름이 존재하지 않는다면 에러를 반환해야 한다. 다음 그림은 위 SQL 문자열의 파싱 트리 구조에서, 앞 장에서 설명한 pt_flat_spec_pre () 단계까지 진행된 트리구조를 나타낸 것이다.



이 트리구조에서 PT_NAME은 세 개가 있다. PT_SPEC 노드와 연결되어 있는 두 개의 PT_NAME은 이전 단계에서 이미 PT_SPEC과 연결되었다. 따라서 이 예제에서는 PT_SPEC과 연결해야 할 PT_NAME 노드는 s_name 이다.

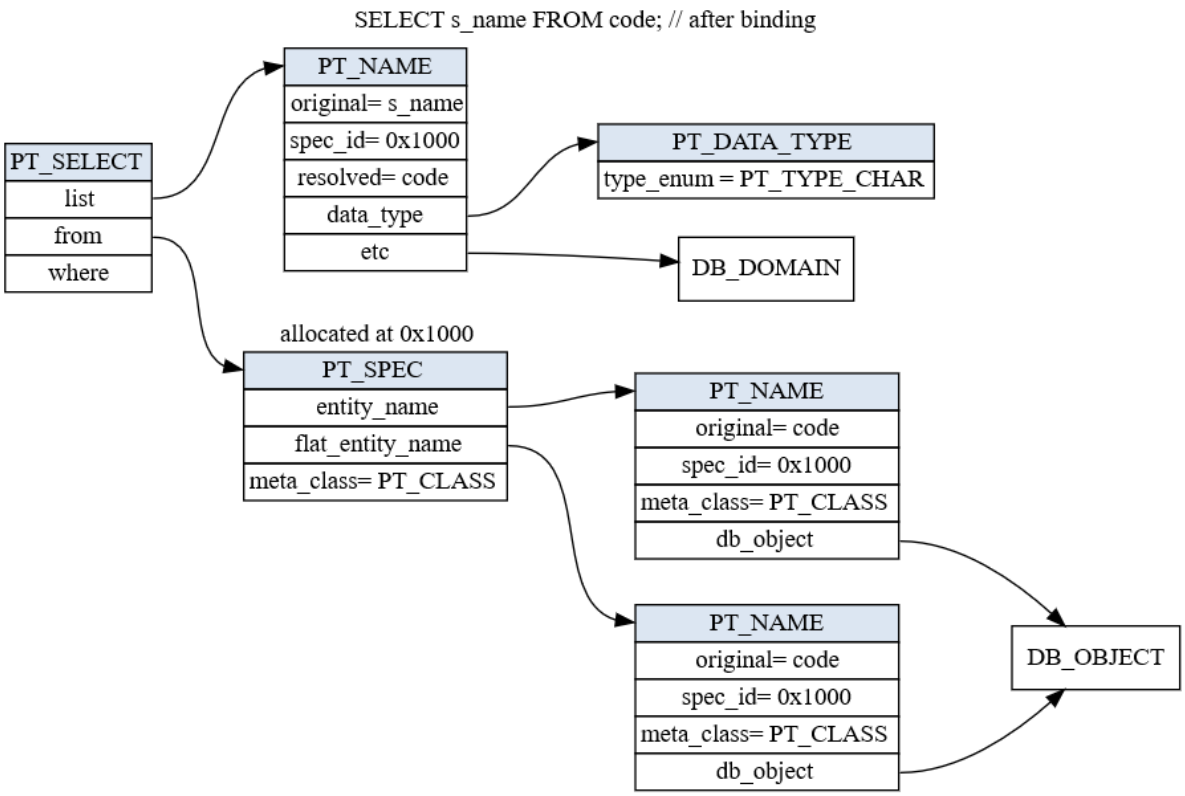
PT_SPEC의 연결 과정은 pt_bind_name_or_path_in_scope () 함수에서 이루어진다. 먼저 s_name 노드에서 볼 수 있는 scope 내에서 PT_SPEC의 정보를 찾아야 한다. 이 scope 정보는 다음 그림과 같은 구조로 관리한다.



name binding을 위해 파싱 트리 전체를 순회하는 동안 하나의 PT_BIND_NAMES_ARG 객체로 관리하며 Scope stack을 구성한다. SCOPES는 Scope stack을 구성하기 위한 Linked list 노드이다. SCOPES는 파싱 트리를 순회하면서 구문을 시작하는 노드 (PT_SELECT, PT_UPDATE 등)를 만날 때 마다 만들어지고, 바깥쪽의 SCOPE를 next에 매달아서 stack으로 관리한다. 즉, 더 좁은 scope가 스택의 맨 위에 위치한다.

s_name 노드를 만나면 이 scope stack에 있는 specs에 연결되어 있는 PT_SPEC 노드로부터 s_name 과 일치하는 테이블의 컬럼 정보가 존재하는지 탐색한다. 탐색에 성공하면 PT_NAME 노드에 다음과 같은 연결된 정보들이 입력된다.

- 연결된 PT_SPEC의 id
- 연결된 PT_SPEC의 이름 (테이블 이름 or alias)
- PT_SPEC의 DB_OBJECT로부터 가져온 컬럼 정보로부터 생성
  - 데이터 타입을 만들어야 되는 경우 PT_DATA_TYPE 노드를 생성
  - DB_DOMAIN 객체를 연결



위 그림은 PT_SPEC과 연결된 정보들이 입력된 후의 PT_NAME 노드를 나타내었다. **spec_id**와 **resolved**에 PT_SPEC의 id 값과 연결된 테이블의 이름이 각각 입력되었다. 그리고 s_name은 문자열 타입의 컬럼이기 때문에 PT_TYPE_CHAR의 **데이터 타입 노드(PT_DATA_TYPE)** 가 만들어진 것을 볼 수 있다.

Resolving *

```
// example 2
SELECT * FROM code;
```

위 SQL 예시처럼 select list에 *가 존재하는 경우 *에 해당하는 부분은 PT_VALUE 타입의 노드로 만들어져 PT_SELECT의 list에 연결된다. 이 단계에서는 PT_VALUE(*) 노드로부터 PT_NAME(column name) 노드의 리스트를 생성한다.

이 때 필요한 정보는 *와 연결된 테이블의 모든 컬럼 정보를 가져와야한다. 즉, 필요한 수행 동작은 다음과 같이 요약할 수 있다.

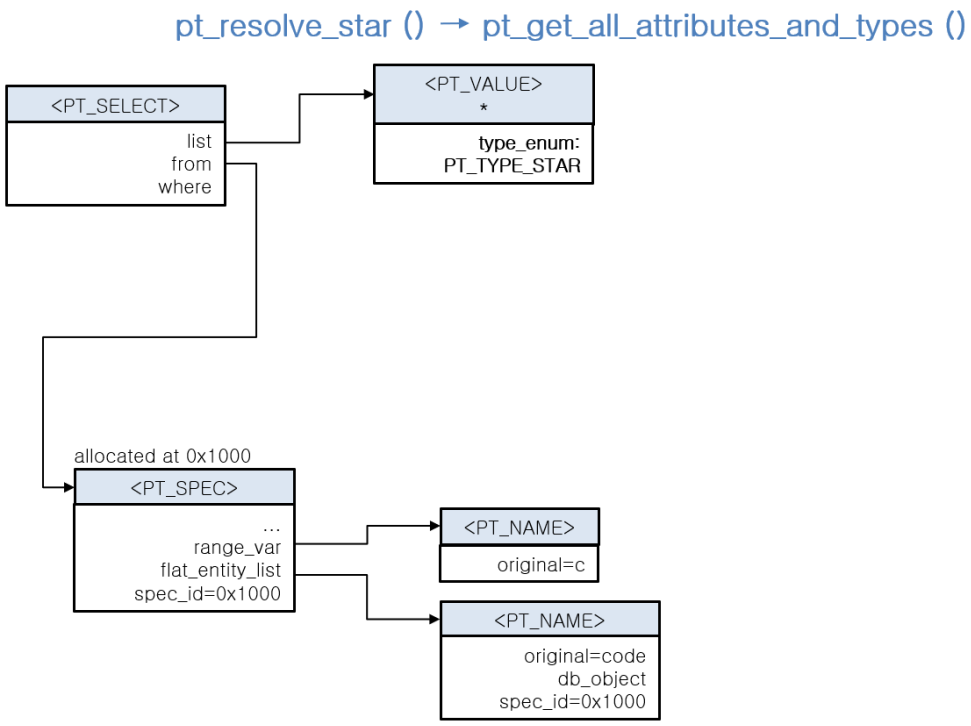
(1) PT_VALUE(*)와 연결할 수 있는 테이블 정보, PT_SPEC을 찾는다.

(2) 테이블이 가지고 있는 모든 컬럼 정보를 가져온다.

(3) 가져온 컬럼 정보로 부터 PT_NAME 노드들을 생성한다.

(4) 새로 생성한 PT_NAME 노드들에 대해 동일하게 name binding을 수행한다.

이 동작이 수행되며 파싱 트리의 구조가 바뀌고 필요한 정보가 입력되는 과정을 다음 그림에서 볼 수 있다. pt_resolve_star () 함수에서 위의 (1) ~ (3) 까지의 과정을 수행하고, 나머지 파싱 트리의 순회 과정 (4)에서 앞 장의 간단한 예시와 동일하게 수행한다.

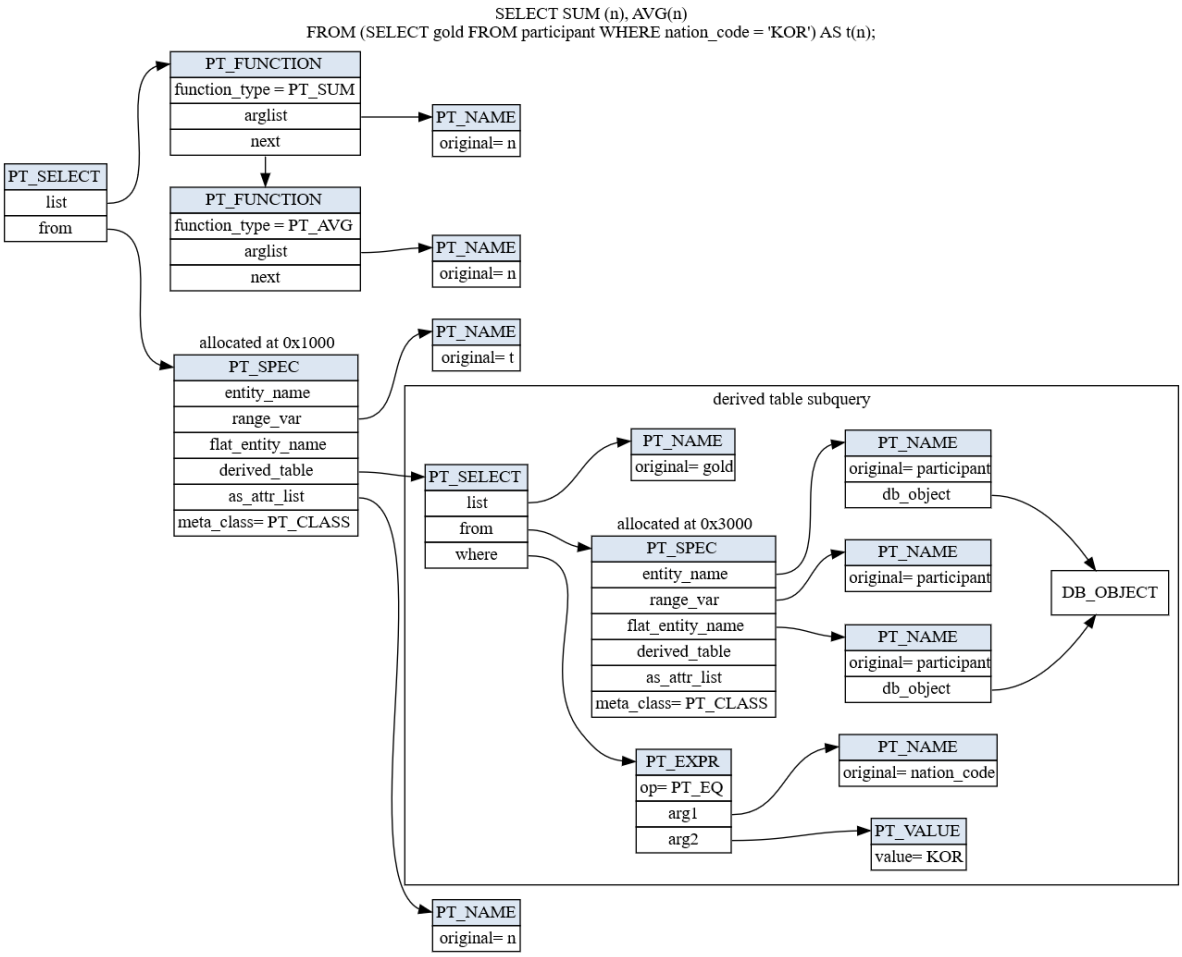


위 결과 그림에서 PT_SPEC 이하의 구조는 앞의 간단한 예시와 같다고 가정한다. PT_SPEC의 flat_entity_list에 연결된 DB_OBJECT로부터 DB_ATTRIBUTE를 가져올 수 있다.

Derived table

```
// subquery derived table syntax
// FROM (subquery)
// [AS] [derived_table_name [(column_name [{, column_name } ... ])]]

// example 3
SELECT SUM (n), AVG(n)
FROM (SELECT gold FROM participant WHERE nation_code = 'KOR') AS t(n);
```



이 예시에서 주목할 부분은 **FROM 절 이후의 subquery**이다. 이 때 AS 구문으로 derived table의 이름(t)와 함께 attribute의 이름(n)을 지정했다. 이때 큐브리드 소스코드에서는 n을 as_attr 라고 부른다.

1) 제일 바깥쪽의 PT_SELECT을 name binding 하는 단계에서 호출하는 pt_bind_scope 함수에서 derived table이 존재한다면 무조건 derived table에 대해 우선적으로 name binding을 먼저 수행한다.

2) 1)의 결과로 derived table 내에서 name binding이 완료되어서 모든 정보가 연결되어 있다.

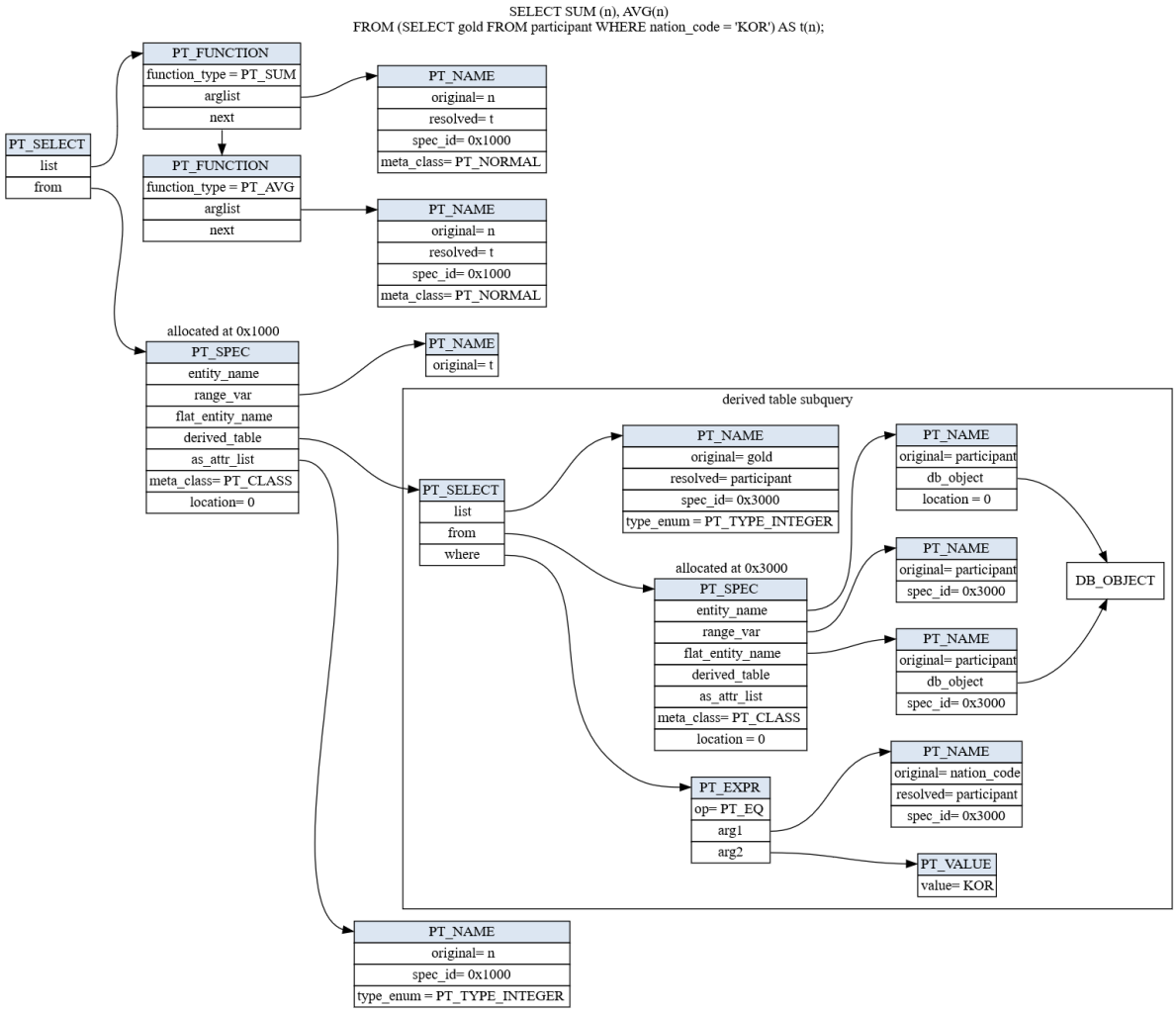
3) pt_semantic_type으로 derived table 내의 노드의 타입을 지정한다.

4) derived table의 select list와 상위 SPEC의 **as_attr_list**를 연결한다.

4)에 대해 자세히 설명하면, 3)에서 derived table 내에서 name binding은 완료된 상태이다. 결국 상위 SPEC에서 필요한 정보는 derived table의 결과 (select list) 인데 이 예제에서는 PT_NAME(gold) 하나이다. 당연히 이 select list의 개수는 이 SELECT를 포함하는 PT_SPEC의 as_attr_list의 길이와 같아야 한다. as_attr_list에 PT_NAME(n)이 하나 존재하므로 갯수가 같아 연결된 것으로 가정한다.
derived table에서 select list에 연결된 gold는 정수 타입으로 PT_INTEGER_TYPE을 type_enum으로 가지는데, 상위 SPEC의 as_attr_list의 n과 **연결**된다. (리스트의 순서대로 그대로 타입을 할당)

> 이 예제의 경우에는 **AS t(n) 구문**에 의해 상위 PT_SPEC에서 as_attr_list에 PT_NAME(n)이 parser 단계에서 생성된다. 만약에 AS t(n) 구문이 존재하지 않으면 **derived table의 select list와 동일한 이름으로 as_attr_list를 새로 생성한다.**

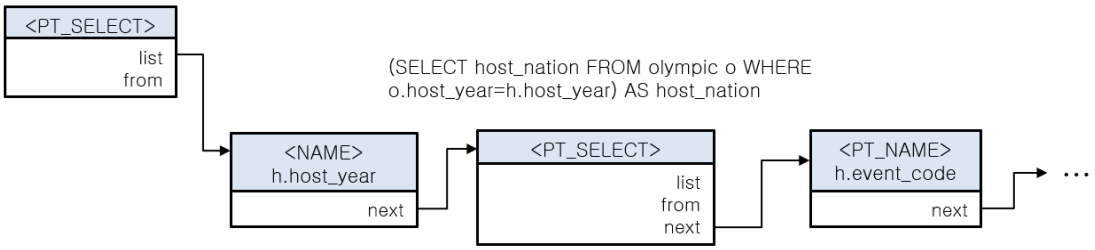
결과는 다음의 그림과 같다



Subquery

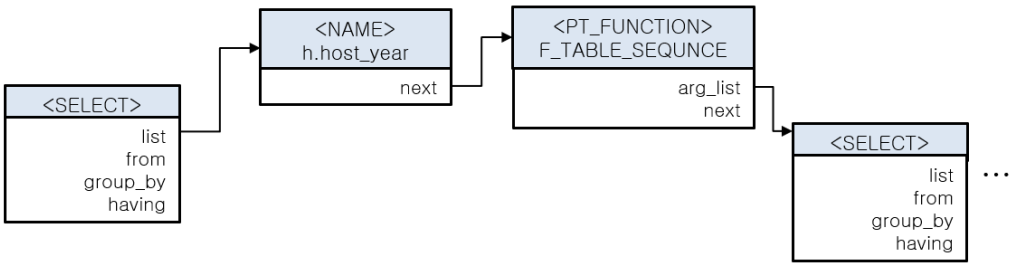
Single-Row Subquery

SELECT h.host_year, **(SELECT host_nation FROM olympic o WHERE o.host_year=h.host_year) AS host_nation**,
h.event_code, h.score, h.unit
FROM history h;



Multi-Row Subquery

SELECT name, capital, **list(SELECT host_city FROM olympic WHERE host_nation = name) AS host_cities**
FROM nation;



Subquery는 select list 또는 where절에 SELECT 구문이 포함된 것이다. derived table과 동일하게 Subquery에 해당하는 SELECT 구문에 대해 먼저 name binding을 수행한다는 것만 참고하면 된다.

Join query

큐브리드에서는 다음의 조인을 지원한다.

- Inner Join
- Outer Join (LEFT, RIGHT including oracle style)
- Natural Join
- Cross Join

```
// join table syntax
/*
<join_table_specification> ::=
    {
        [INNER | {LEFT | RIGHT} [OUTER]] JOIN
     } <table_specification> ON <search_condition>
```

```
<join_table_specification2> ::=
            {
        CROSS JOIN |
        NATURAL [ LEFT | RIGHT ] JOIN
    } <table_specification>
*/
// example 4-1 (inner join with on condition) - ANSI
SELECT DISTINCT h.host_year, o.host_nation
FROM history h INNER JOIN olympic o ON h.host_year = o.host_year
WHERE o.host_year > 1950;

// example 4-2 (inner join condition in where clause)
SELECT DISTINCT h.host_year, o.host_nation
FROM history h, olympic o WHERE h.host_year = o.host_year AND o.host_year > 1950;

// example 4-3 (outer join with on condition) - ANSI
SELECT DISTINCT h.host_year, o.host_nation
FROM olympic o LEFT OUTER JOIN history h ON h.host_year = o.host_year
WHERE o.host_year > 1950;

// example 4-4 (oracle style outer join)
SELECT DISTINCT h.host_year, o.host_nation
FROM history h, olympic o
WHERE o.host_year = h.host_year(+) AND o.host_year > 1950;

// example 4-5 (natural join)
SELECT DISTINCT h.host_year, o.host_nation
FROM history h NATURAL JOIN olympic o WHERE o.host_year > 1950;
```

Join을 사용할 때 다음과 같이 표준에 맞는 ANSI 문법으로 작성할 수도 있고, Conventional한 방법으로 작성할 수도 있다.

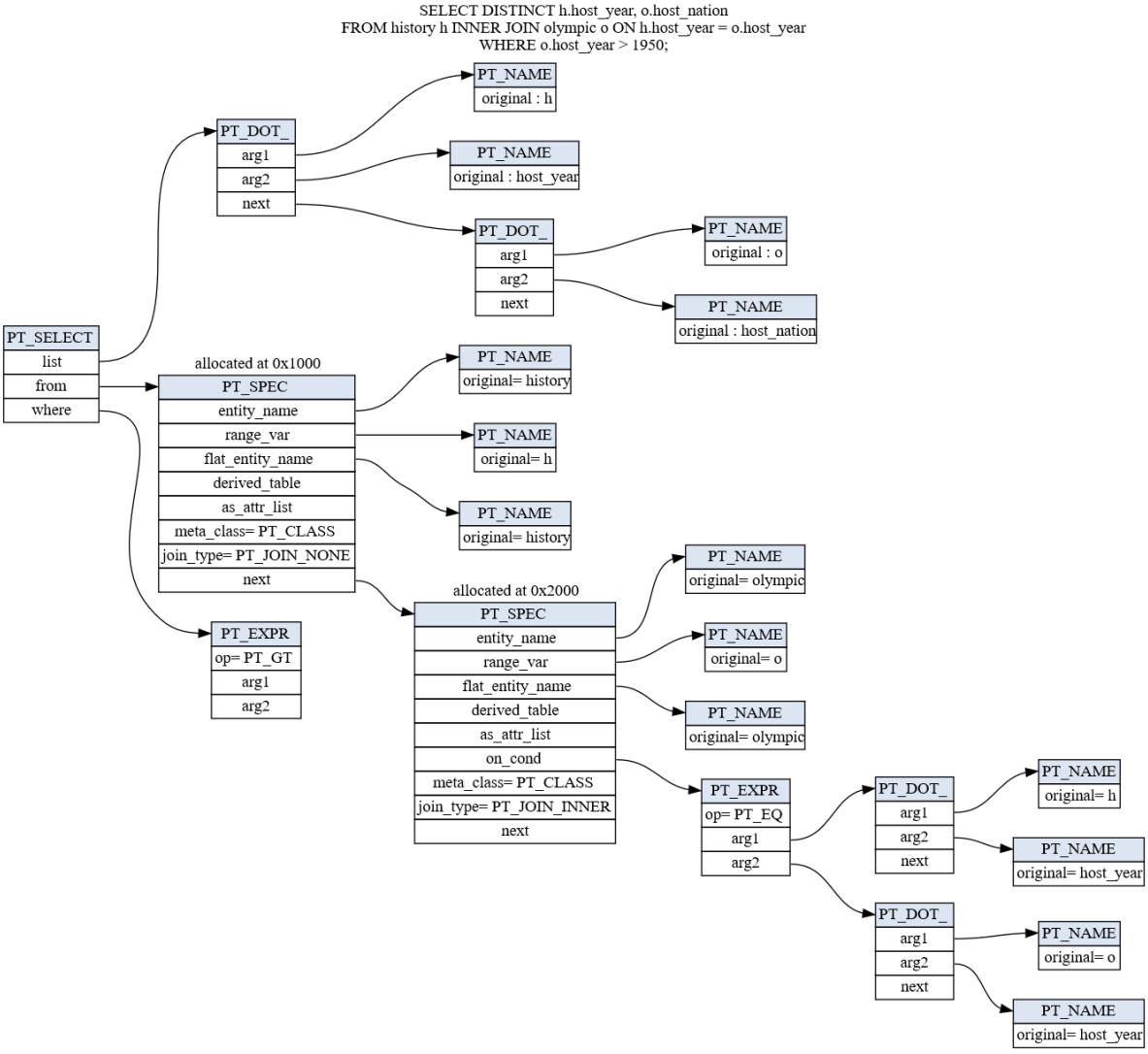
```
┌──────────────────────────────────┬──────────────────────────────────────────┐
│ INNER JOIN - CONVENTIONAL        │ INNER JOIN - ANSI SYNTAX                 │
├──────────────────────────────────┼──────────────────────────────────────────┤
│ SELECT DISTINCT                  │ SELECT DISTINCT                          │
│      h.host_year, o.host_nation  │      h.host_year, o.host_nation          │
│ FROM                             │ FROM                                     │
│      history h, olympic o        │     history h INNER JOIN olympic o       │
│ WHERE                            │     ON h.host_year = o.host_year         │
│      h.host_year = o.host_year   │ WHERE                                    │
│      AND o.host_year > 1950;     │     o.host_year > 1950;                  │
├──────────────────────────────────┼──────────────────────────────────────────┤
│ LEFT OUTER JOIN - CONVENTIONAL   │ LEFT OUTER JOIN - ANSI SYNTAX            │
├──────────────────────────────────┼──────────────────────────────────────────┤
│ SELECT DISTINCT                  │ SELECT DISTINCT                          │
│      h.host_year, o.host_nation  │      h.host_year, o.host_nation          │
│ FROM                             │ FROM                                     │
│      history h, olympic o        │     history h LEFT OUTER JOIN olympic o  │
│ WHERE                            │     ON h.host_year = o.host_year         │
│      h.host_year = o.host_year(+) │ WHERE                                    │
│      AND o.host_year > 1950;     │     o.host_year > 1950;                  │
├──────────────────────────────────┼──────────────────────────────────────────┤
│ RIGHT OUTER JOIN - CONVENTIONAL  │ RIGHT OUTER JOIN - ANSI SYNTAX           │
├──────────────────────────────────┼──────────────────────────────────────────┤
│ SELECT DISTINCT                  │ SELECT DISTINCT                          │
│      h.host_year, o.host_nation  │      h.host_year, o.host_nation          │
│ FROM                             │ FROM                                     │
│      history h, olympic o        │     history h RIGHT OUTER JOIN olympic o │
│ WHERE                            │     ON h.host_year = o.host_year         │
│      h.host_year(+) = o.host_year │ WHERE                                    │
│      AND o.host_year > 1950;     │     o.host_year > 1950;                  │
├──────────────────────────────────┼──────────────────────────────────────────┤
```

만약 SQL 문자열이 ANSI 표준에 맞게 작성되어있다면 parser에서 분석되어 join table spec 문법에 따라 어떤 Join 타입인지 결정된다. 각 Join 타입에 대해 PT_SPEC 노드의 join_type 변수에 지정된다. 지정되는 enum 타입은 다음과 같다. 이 때 만약 A join B 와 같다면 B에 대한 SPEC에만 Join 타입이 지정되고 A에는 PT_JOIN_NONE이 지정된다.

```
/* Enumerated join type */
typedef enum
{
  PT_JOIN_NONE = 0x00,    /* 0000 0000 */
  PT_JOIN_CROSS = 0x01,   /* 0000 0001 */
  PT_JOIN_NATURAL = 0x02, /* 0000 0010 -- not used */
  PT_JOIN_INNER = 0x04,   /* 0000 0100 */
  PT_JOIN_LEFT_OUTER = 0x08,  /* 0000 1000 */
  PT_JOIN_RIGHT_OUTER = 0x10, /* 0001 0000 */
  PT_JOIN_FULL_OUTER = 0x20,  /* 0010 0000 -- not used */
  PT_JOIN_UNION = 0x40    /* 0100 0000 -- not used */
} PT_JOIN_TYPE;
```

Full Outer Join은 정의해놓았으나 지원하지 않는다. 그리고 Natural Join의 경우PT_JOIN_NATURAL 을 사용하지 않고 SQL에서 NATURAL이 지정된 경우 PT_SPEC 노드의 natural 변수에 true를 저장하여 표시한다.

ANSI Inner join, outer join



ANSI 표준 문법으로 작성된 경우 join 조건이 ON 절, 아닌 경우 WHERE 절에 연결된다. ANSI 문법으로 작성된 Inner Join 예시의 파싱 트리는 위 그림과 같다.

살펴보면 앞에서 소개하지 않은 두 가지 다른점이 존재한다. 하나씩 살펴보자.

1) 먼저 PT_DOT_ 노드이다. 이전 예제에서는 PT_SELECT의 list에 PT_NAME이 링크드리스트 형태로 연결되어 select list를 구성하였다. 이 예제에서는 select list에 h.host_year, o.host_nation와 같이 점을 기준으로 앞은 SPEC 정보, 뒤는 컬럼 정보로 이루어져 있는데, 이 구조를 나타낸 노드이다.

> 💡 큐브리드는 ORDBMS이기 때문에 A.B.C.D.E.col1과 같이 컬럼 정보를 나타내는 것이 가능하다. 그러나 여기서는 PT_DOT_이 <tbl_alias>.<col_name> 구조로만 이루어져 있다고 가정하고 설명한다.

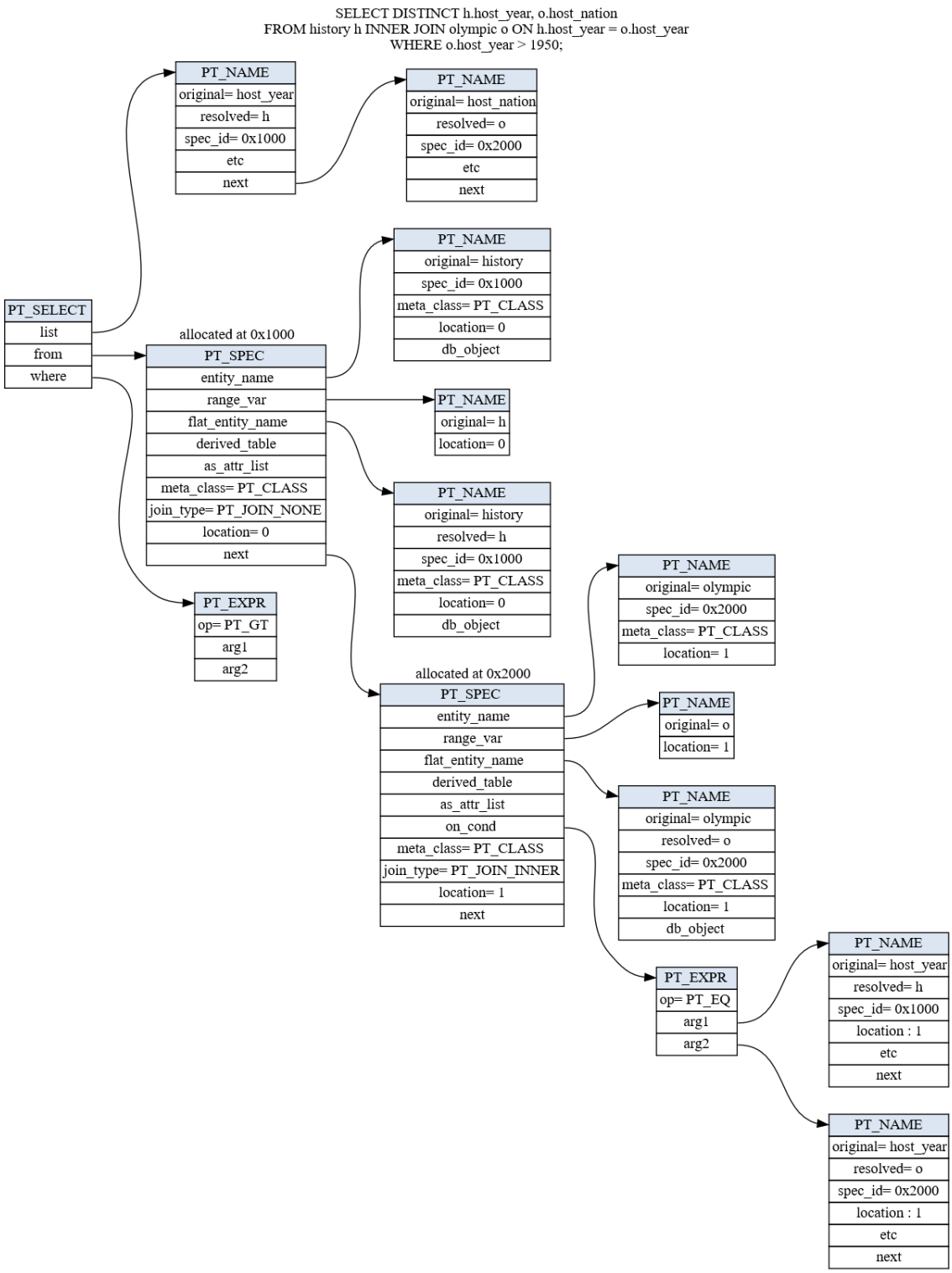
2) 또한 여기서 주목해야 할 것은 **두 개의 PT_SPEC이 next로 연결되어 있는 것이다.** 각 PT_SPEC은 FROM 절에서 주어진 각각의 테이블 (history, olympic)을 나타내고, 순서대로 연결되어 있다.

1) PT_DOT_

PT_DOT_ 노드는 arg1과 arg2에 각각 PT_NAME 노드를 가지고 있다. name resolution 단계에서 PT_DOT_ 노드를 포함한 서브트리는 하나의 PT_NAME으로 변환된다.

2) PT_SPEC의 연결

join spec이 존재하면 PT_SPEC은 next로 연결된다. location 값은 next로 연결된 순서대로 0, 1, 2와 같이 1씩 더해가며 설정된다. 기본값은 0이다. 이 때 join 조건을 포함한 PT_SPEC에 연결된 하위 노드들 (PT_EXPR, PT_NAME...)에 대해 동일한 location 값을 모두 할당한다.



그 결과는 위의 그림과 같다. 이 예시의 경우 ANSI Syntax를 가진 Join이기 때문에 on_cond 내에서 location이 1로 설정되어 있다. 만약 Conventional syntax를 가진 경우에는 join 조건이 where 절에 위치한다. 그리고 location이 따로 할당되지 않고 기본값인 0으로 설정된다.

이 단계에서는 oracle style outer join 유무 외에는 어떤 join인지, 표준에 맞는 syntax로 쓰여졌는지는 고려하지 않는다. 따라서 ANSI inner join, outer join 그리고 conventional inner join에 대해서는 동일한 로직이 수

행된다.

다음으로는 oracle style의 outer join인 경우에는 inner join 또는 ANSI outer join으로 변환되는 과정을 설명한다.

---

oracle style outer join

Oracle style outer join은 join 조건에서 다음과 같은 모양의 predicate를 가질 수 있다.

```
single_column(+) op expression //right
expression       op single_column(+) //left
```

이 단계에서는 WHERE 절에 join 조건에 해당하는 이 predicate를 FROM 절의 on_cond로 옮기고 적절한 join type을 지정하려고 한다.

파싱 단계에서

single_column(+) op expression
expression op **single_column(+)**

위 모양을 저장하고 있는 PT_EXPR 노드에 각각 PT_EXPR_INFO_RIGHT_OUTER 또는 PT_EXPR_INFO_LEFT_OUTER가 flag에 설정된다.

```
opt_paren_plus
    {{

  /* 앞에서 (+)가 있었다면 RIGHT OUTER JOIN, 없었다면 NONE */
  PT_JOIN_TYPE join_type = parser_pop_join_type ();
  PT_NODE *e, *attr;

  if ($3)
    {
      if (join_type == PT_JOIN_RIGHT_OUTER)
        join_type = PT_JOIN_FULL_OUTER;
      else
        join_type = PT_JOIN_LEFT_OUTER;
    }
```

SELECT 문에서 ORACLE OUTER JOIN이 존재하면, PT_SELECT 노드에 **PT_SELECT_INFO_ORACLE_OUTER flag가 설정**된다. 만약 이 flag가 설정되어 있다면 Oracle style의 outer join을 ANSI 문법에 맞는 join으로 대체하기 위한 로직을 수행한다.

수행하려는 로직은 다음 그림의 설명과 같다.

이를 위해 전체적으로 2-phase의 검색을 거친다.

List search twice

- first-pass: set outer join type, edge

1) outer join_link handling

2) set join type

oracle style outerjoin은 join 조건이 where절에 존재한다. where 절에서 (+)가 있는 predicate를 찾으면, 이것으로부터 lhs, rhs로 나누어 가져온다. lhs와 rhs에서 JOIN에 참가하는 attribute를 찾아 lhs_spec과 rhs_spec을 찾았으면 각 PT_SPEC의 location으로 순서를 본다. 만약에 그 순서가 뒤집어져 있다면 join type을 재조정한다.

```
if (lhs_spec && rhs_spec && lhs_spec->info.spec.id != rhs_spec->info.spec.id)
{
  /* found edge: set join type and spec */
  if (lhs_location < rhs_location) // 순서가 제대로?
  {
  p_spec = lhs_spec;
  spec = rhs_spec;
  }
  else /* SELECT...  FROM x, y WHERE y.i = x.i(+) */
  {
  join_type = (join_type == PT_JOIN_LEFT_OUTER) ? PT_JOIN_RIGHT_OUTER : PT_JOIN_LEFT_OUTER;
  p_spec = rhs_spec;
  spec = lhs_spec;
  }
}
```

p_spec과 spec을 찾았으면 이제 p_spec과 spec의 범위가 어디인지 알아낸다.

TBU (분석이 완료되면 추가할 예정)

- Meaningless outer join check

```
SELECT... FROM x, y ON y.i(+) = 1

=> SELECT... FROM x, y WHERE x.i(+) = 1
```

과 같은 구문의 경우에는 JOIN_TYPE이 PT_JOIN_NONE이 된다. (실제로 JOIN이 되지 않는 구문이기 때문) 따라서 on_cond에 매달린 expression을 다시 WHERE 절로 옮긴다.

Group by / Having Alias

파싱 노드를 순회하며 GROUP BY나 HAVING 절 안의 PT_EXPR이나 PT_SORT_SPEC에 속한 모든 PT_NAME 노드를 찾는다. 이 단계에서 주요 로직은 찾은 PT_NAME에 대해서 만약 select list와 동일한 이름의 alias가 있다면 기존에 있는 group by나 having의 PT_NAME 노드를 지우고 select list의 PT_NAME 노드와 동일한 것을 복사하여 기존의 자리에 붙이는 것으로 alias를 연결한다.

다음은 pt_resolve_group_having_alias_pt_name() 함수의 주요 로직이다.

```
for (col = select_list; col != NULL; col = col->next)
    {
      if (col->alias_print != NULL && intl_identifier_casecmp (n_str, col->alias_print) == 0)
  {
    parser_free_node (parser, *node_p);
    *node_p = parser_copy_tree (parser, col);
    if ((*node_p) != NULL)
      {
        (*node_p)->next = NULL;
      }
    break;
  }
    }
```

소스 참조: pt_resolve_group_having_alias ()

Natural join resolution

```
static PT_NODE *
pt_resolve_natural_join (PARSER_CONTEXT * parser, PT_NODE * node, void *chk_parent, int *continue_walk)
{
  PT_NODE *join_lhs, *join_rhs;

  *continue_walk = PT_CONTINUE_WALK;

  if (node == NULL || node->node_type != PT_SPEC)
    {
      return node;
    }

  // 트리 전부를 탐색하지만, 결국 PT_SPEC에서만 실행됨
  join_lhs = node;
  join_rhs = node->next;

  /* there is a natural join */
  if (join_rhs != NULL && join_rhs->node_type == PT_SPEC && join_rhs->info.spec.natural == true)
    {
      pt_resolve_natural_join_internal (parser, join_lhs, join_rhs);
    }
```

```
  return node;
}
```

A_SPEC NATURAL JOIN B_SPEC에서 A_SPEC과 B_SPEC의 attribute를 각각 가져온다. attribute 정보를 가져오는 경우는 구문에 따라 다음의 두가지 방법이 있다.

case 1) entity로 부터 가져오는 경우

case 2) derived table이나 CTE

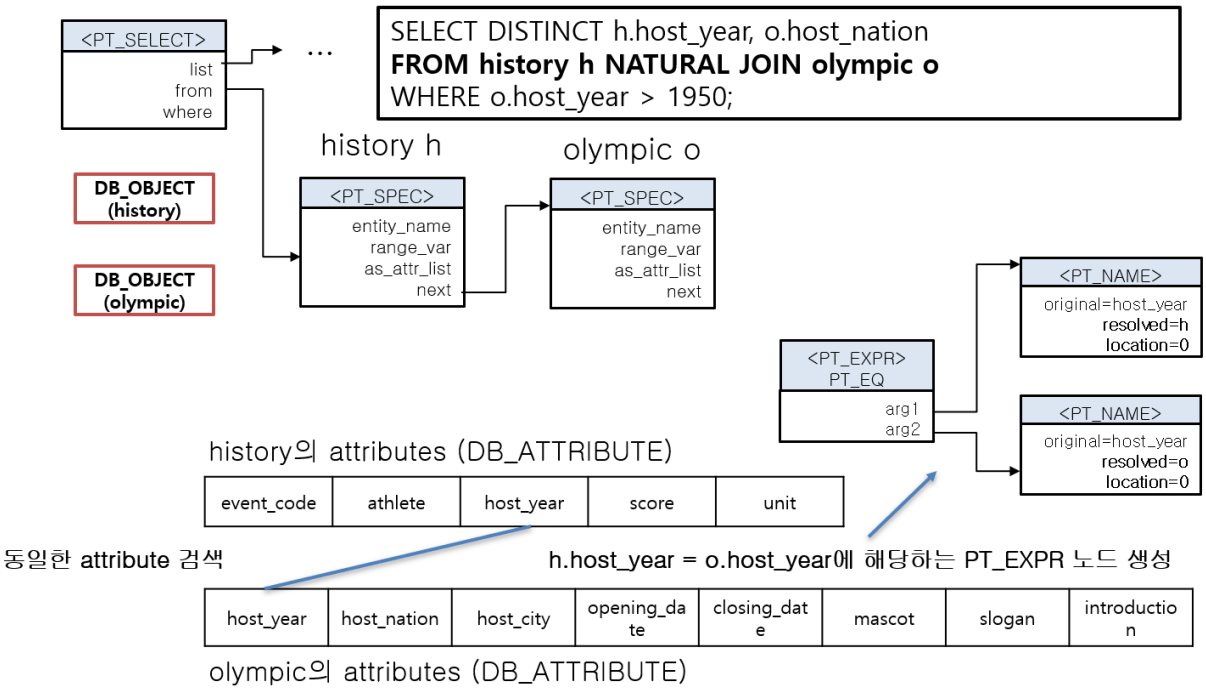
entity의 경우 SPEC에 연결된 DB_OBJECT로부터 DB_ATTRIBUTE를 취득하여 attribute 이름들을 가져올 것이고, derived table이나 CTE의 경우에는 select list나 with 절에 포함된 PT_NAME의 리스트로 부터 attribute 이름을 만들어낼 것이다.

위에서 생성한 두개의 PT_NAME의 리스트로부터 A_SPEC이 가진 PT_NAME 리스트와 B_SPEC의 PT_NAME 리스트 비교해서 이름이 같은 경우를 찾는다. 이름이 같은 PT_NAME을 찾는다면, A_SPEC.name = B_SPEC.name과 같은 모양의 PT_EXPR 노드를 새로 추가한다. 그리고 B_SPEC의 on_cond에 추가한다.

예시)

```
SELECT DISTINCT h.host_year, o.host_nation
FROM history h NATURAL JOIN olympic o
WHERE o.host_year > 1950;


// rewritten query
SELECT DISTINCT h.host_year, o.host_nation
FROM history h INNER JOIN olympic o ON h.host_year = o.host_year // JOIN PRECIATE 추가
WHERE o.host_year > 1950;
```



FOR UPDATE 절 flag 설정

```
SELECT ... [FOR UPDATE [OF <spec_name_comma_list>]]

    <spec_name_comma_list> ::= <spec_name> [, <spec_name>, ... ]
        <spec_name> ::= table_name | view_name
```

FOR UPDATE 구문은 spec_name_comma_list에 명시되는 테이블 또는 뷰에 대해 락을 잡거나, 생략된 경우 FROM 절에서 참조되는 모든 테이블 또는 뷰에 대해 락을 잡는다. (X_LOCK)

이 단계에서는 PT_SELECT 노드에 FOR_UPDATE가 포함되어있다는 **flag**만을 설정하고 넘어간다.

flag: PT_SELECT_INFO_FOR_UPDATE